



Welcome to A Catholic Introduction to Artificial Intelligence Class 5! Deep Learning and Large Language Models (LLMs)

"AI is a tool for discipline, not a substitute for it."

Learning Objectives

By the end of this class, you will be able to explain what deep learning is, understand the basic idea behind large language models (LLMs), and recognize both the strengths and limitations of LLM systems like ChatGPT.

1. Class 4 Review

Class 4 introduced one of the most important ideas in modern artificial intelligence:

machine learning. Unlike traditional programming, where humans explicitly write rules for a computer to follow, machine learning allows a system to learn patterns directly from data. You explored how this shift changed the field of AI by making it possible for computers to recognize speech, classify images, recommend content, and detect fraud without programmers specifying every possible rule ahead of time.

The class also examined the difference between supervised and unsupervised learning. In supervised learning, models are trained using labeled data, meaning the correct answers are already known during training. Examples discussed included spam detection and image classification. In unsupervised learning, the system receives unlabeled data and attempts to

discover patterns or groupings on its own. You learned that many modern AI systems rely heavily on these two approaches to process enormous amounts of information efficiently.

Another major focus of the lesson was neural networks. You were introduced to the idea that neural networks are loosely inspired by the structure of the human brain and are made up of layers of interconnected "neurons." Each neuron processes information and passes it to the next layer, allowing the system to gradually recognize increasingly complex patterns. Although the mathematical details were simplified, you learned that neural networks improve by adjusting internal weights during training to make more accurate predictions.

The lesson emphasized that modern AI systems are primarily powerful pattern-recognition systems rather than truly human-like thinkers. Students discussed how machine learning systems can appear intelligent because they are trained on massive datasets and can identify relationships humans might miss. At the same time, the class introduced the idea that the quality of an AI system depends heavily on the quality of its data, which connects directly to future discussions about bias, fairness, and ethics in AI.

Finally, you explored real-world applications of machine learning and neural networks through demonstrations and examples. Systems such as recommendation engines, voice assistants, and image recognition tools illustrated how these technologies are already deeply integrated into everyday life. The key takeaway from the lesson was that modern AI works not because programmers wrote every rule manually, but because computers can now learn patterns from data at a scale never before possible.

A Small Example: Training on a Single Row

How does a network actually "adjust its weights"? Let's slow everything down and watch one neuron learn from one row of our homework data. It will make a guess, flip the flashcard over to check the label, measure how wrong it was, and nudge its weights. Then it repeats.

```
In [1]: import math
import pandas as pd

hw_df = pd.read_csv('synthetic_homework_dataset.csv')
```

```
In [2]: # 1. Pick ONE row (one "flashcard") from the dataset
row = hw_df.iloc[1]
x = [float(row["prior_completion_rate"]),          # already 0-1
     float(row["time_spent_minutes"]) / 100,      # scale to roughly 0-1
     float(row["difficulty"]) / 5]                # scale 1-5 down to 0-1
label = int(row["completed_on_time"])              # the correct answer (1 = on time)
print("Features (inputs):", [round(v, 2) for v in x])
print("Label (correct answer):", label)

# 2. A single neuron starts with guessed weights
weights = [0.5, 0.5, 0.5]
bias = 0.0
learning_rate = 0.5
```

```
def predict(x, weights, bias):
    total = sum(w * xi for w, xi in zip(weights, x)) + bias    # W1X1 + W2X2 + W3X3
    return 1 / (1 + math.exp(-total))                        # activation h(): squ

# 3. Train on this one row a few times: guess, check the answer, adjust
for step in range(1, 6):
    guess = predict(x, weights, bias)
    error = guess - label                                     # how wrong was the g
    weights = [w - learning_rate * error * xi for w, xi in zip(weights, x)]
    bias = bias - learning_rate * error
    print(f"Step {step}: guess = {guess:.2f} error = {error:+.2f} "
          f"new weights = {[round(w, 2) for w in weights]}")

print(f"\nAfter training, the neuron's guess for this row: {predict(x, weights, bia
```

Features (inputs): [0.8, 0.66, 1.0]

Label (correct answer): 0

Step 1: guess = 0.77 error = +0.77 new weights = [0.19, 0.25, 0.11]

Step 2: guess = 0.51 error = +0.51 new weights = [-0.01, 0.08, -0.14]

Step 3: guess = 0.32 error = +0.32 new weights = [-0.14, -0.03, -0.3]

Step 4: guess = 0.22 error = +0.22 new weights = [-0.23, -0.1, -0.42]

Step 5: guess = 0.17 error = +0.17 new weights = [-0.3, -0.16, -0.5]

After training, the neuron's guess for this row: 0.14

Observation(s):

- At first, the neuron guessed 0.77, meaning "probably on time." The label says this assignment was late (0), so the error was large.
- Each step, the weights moved in the direction that shrinks the error. The size of each change depended on how wrong the guess was. Big mistakes produce big corrections, and small mistakes produce small ones.
- The **difficulty** weight moved the most. For this row, the high difficulty was the strongest clue that the work would be late.
- A real model does this for all 400 training rows, many times over. It is learning a pattern across thousands of flashcards, not just this one. If we trained only on a single row, the model would simply memorize it, which is the overfitting problem from Section 4.

2. What is Deep Learning?

Deep Learning is a branch of machine learning that teaches computers to recognize patterns and make decisions by using math that mimics human brain's ability to recognize patterns. At its core are artificial neural networks (ANNs) — layers of interconnected mathematical "neurons" that transform input data (like images, sound, or text) into useful outputs (such as labels, translations, or answers). What makes deep learning "deep" is the use of many stacked layers; each layer extracts progressively higher-level features. **For**

example, when a network analyzes a photograph, early layers detect edges and colors, middle layers detect textures and shapes, and deeper layers recognize objects like faces or cars.

Training a deep learning model means adjusting the connections (weights) between neurons (or nodes) so the network's outputs match desired results. This is done with large datasets and a process called **backpropagation**, where the model measures its mistakes, computes how much each weight contributed to those mistakes, and nudges weights to improve future performance. Because deep networks have millions or even billions of parameters, they can learn very complex patterns—but they also need lots of data and computing power to do it well.

Deep learning excels in tasks that were historically very hard for computers: image recognition, speech recognition, translation, and natural language understanding. Modern breakthroughs — like the systems that power image tagging on phones or the language models behind chatbots — are largely deep learning successes. Those systems can generalize from examples, pick up subtle correlations, and produce outputs that often feel intelligent.

However, deep learning has limits. Models learn statistical patterns in their training data; **they don't possess understanding, intentions, or moral awareness.** They can make confident-sounding errors, reproduce biases present in their training data, or fail when faced with situations unlike their training examples. Interpreting why a deep model made a decision can be difficult, which raises questions about accountability and fairness.

From a Catholic perspective, deep learning is a powerful tool that reflects human creativity and the drive to steward creation responsibly. It offers real goods: medical image analysis that can assist doctors, accessibility tools that help people with disabilities, and educational aids that support learning. At the same time, moral discernment is necessary: we must guard human dignity, prevent injustice caused by biased systems, and ensure these technologies serve the common good rather than undermine it. Teachers and developers carry a responsibility to design and deploy deep learning systems ethically.

In summary, deep learning is a method for building layered neural networks that learn from data to solve complex problems. It is transformative and practical, yet not magical—its strengths come with technical and ethical trade-offs. For this class, we'll combine hands-on experiments with thoughtful discussion so students can both see how deep learning works and reflect on how it should be used in a world guided by reason and moral principles.

Let's take a look at a deep learning example using a brain tumor case study.

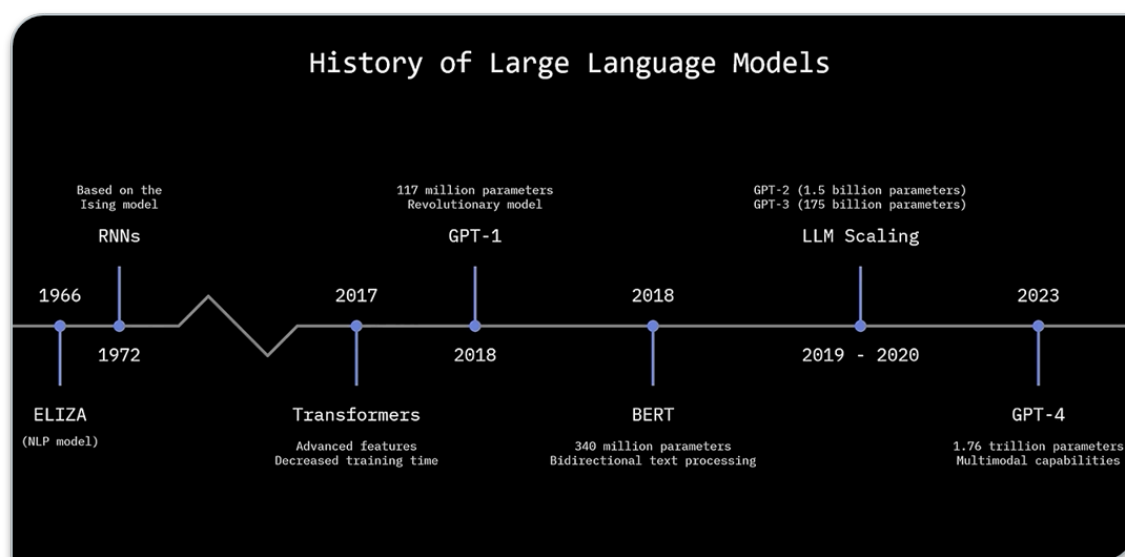
3. What are LLMs?

Large Language Models (LLMs) are advanced AI systems trained to understand and generate human language. Built on deep learning techniques—especially large neural networks—LLMs learn language patterns by analyzing massive amounts of text from books, websites, and other written sources. The “large” in LLM refers both to the size of the training data and the number of parameters (the model’s internal adjustable values). Bigger models with more parameters can capture more subtle patterns of grammar, meaning, and context, allowing them to produce fluent, often impressive text.

Training an LLM involves two main stages. First, the model is pre-trained on vast corpora of text using self-supervised learning: it practices predicting missing words or the next words in a sentence. This stage teaches the model syntax, common facts, and many language patterns without needing labeled examples. Second, models are often fine-tuned on narrower datasets or with human feedback to make them safer, more accurate, or better at specific tasks like answering questions or following instructions. Fine-tuning helps align the model’s behavior with human expectations.

What can LLMs do? They can write essays, summarize articles, translate languages, answer questions, draft emails, generate creative stories or code, and simulate a conversational partner. They are used in chatbots, writing assistants, tutoring tools, and customer service systems. The apparent fluency of LLM responses comes from their statistical mastery of language: they predict what words are likely to come next given the context, which often results in coherent and relevant outputs.

In short, LLMs are powerful language-generating AI systems built with deep learning. They offer remarkable capabilities and practical benefits, yet they come with technical shortcomings and ethical responsibilities. In class, we’ll interact with LLMs to test their strengths and weaknesses and discuss how to use them wisely in learning and life.



Note: OpenAI never published the parameter numbers; they are estimates.

3.1 How Do LLMs Work?

This demo demonstrates a toy implementation of how an LLM generates text (autoregressive generation). It follows six core steps: **tokenization, embeddings, attention/context, computing logits (with a logit bias), sampling/decoding, and the autoregressive loop with a stop condition.**

We use simplified math and tiny embeddings so we can run and inspect each step. The notebook also shows how a logit bias can nudge the model to produce a target phrase ("have mercy on us.") and explains why that is ethically and technically interesting.

Autoregressive generation is a step-by-step process where a language model produces text one token at a time, always predicting the next token conditioned on everything generated so far. Because each choice depends on the growing prefix, small early decisions shape the whole output making the model excellent at fluent continuations but also sensitive to prompt wording and sampling settings.

A **logit** is an unnormalized score the model produces for each possible next token before those scores are converted into probabilities. After computing logits, the model applies **softmax** - (function that converts a set of numbers (logits) into probabilities that add up to 1) to turn them into a probability distribution, so higher logits mean a token is more likely to be chosen.

3.2 What Makes LLMs So Conversational

Training on dialogue patterns. Models like Claude are trained on huge amounts of text that includes back-and-forth exchanges, so they learn the rhythm of turn-taking, follow-up questions, and context-dependent replies—not just isolated facts.

Context window. Most models hold the entire conversation in memory as it generates each response, so it can refer back to what you said three messages ago, catch a pronoun's antecedent, or adjust tone based on how you're writing.

Fine-tuning for helpfulness and instruction-following. After initial training, models go through additional stages (like **RLHF—reinforcement learning** from human feedback) where humans rate responses for being clear, relevant, and appropriately conversational, which shapes the model away from robotic or generic-sounding text.

Next-token prediction, at scale. Under the hood, it's still predicting the most likely next word given everything before it—but at the scale modern models operate, that prediction captures enough nuance (tone, intent, implication) that the output reads as genuinely responsive rather than templated.

Let's take a look at how this might work.

```
In [3]: # Install numpy. numpy is a fundamental package for scientific computing in Python.

%pip install numpy
```

Requirement already satisfied: numpy in d:\career\contracting\hsc\catholic-intro-to-ai\.venv\lib\site-packages (2.4.5)

Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 24.0 -> 26.2.1

[notice] To update, run: python.exe -m pip install --upgrade pip

```
In [4]: import numpy as np
import random
from math import exp

# Toy tokenized input and embeddings
tokenized_sentence_2 = ["Lord", "Jesus", "Christ", "Son", "of", "the", "Living", "G
token_to_id = {
    "Lord": 102, "Jesus": 203, "Christ": 458, "Son": 71, "of": 5, "the": 3,
    "Living": 890, "God": 412, ",": 12
}

# Create the embedding matrix for the tokenized sentence using the token_to_id mapp
embedding_matrix = {
    102: np.array([0.21, -0.11, 0.54, 0.03]),
    203: np.array([0.19, -0.09, 0.58, 0.01]),
    458: np.array([0.22, -0.12, 0.60, 0.00]),
    71: np.array([0.05, 0.02, 0.10, 0.01]),
    5: np.array([0.01, 0.00, 0.02, 0.00]),
```

```

3:    np.array([0.02, -0.01, 0.01, 0.00]),
890: np.array([0.18, -0.05, 0.30, 0.02]),
412: np.array([0.20, -0.10, 0.55, 0.02]),
12:  np.array([0.00, 0.00, 0.00, 0.00]),
}

# Candidate tokens including target phrase tokens. These are for generating possible
extra_tokens = {
    "<STOP>": 999, "reigns": 1000, "is": 1001, "Lord.": 1002, "blessed": 1003,
    "have": 1004, "mercy": 1005, "on": 1006, "us.": 1007
}

# Create the embedding matrix for the tokenized sentence using the token_to_id mapping
extra_embeddings = {
    999: np.array([0.00, 0.00, 0.00, 0.00]),
    1000: np.array([0.15, -0.07, 0.40, 0.02]),
    1001: np.array([0.03, -0.01, 0.05, 0.00]),
    1002: np.array([0.20, -0.10, 0.56, 0.02]),
    1003: np.array([0.10, -0.02, 0.25, 0.01]),
    1004: np.array([0.05, -0.01, 0.12, 0.00]),
    1005: np.array([0.09, -0.02, 0.20, 0.01]),
    1006: np.array([0.02, 0.00, 0.03, 0.00]),
    1007: np.array([0.03, -0.01, 0.05, 0.00]),
}

# Update the main embedding matrix with the extra embeddings for candidate tokens.
embedding_matrix.update(extra_embeddings)
id_to_token = {v: k for k, v in token_to_id.items()}
id_to_token.update({v: k for k, v in extra_tokens.items()})
token_to_id.update(extra_tokens) # ensure reverse lookup exists for generated tokens
candidate_ids = list(extra_embeddings.keys())

# Utility functions
# Cosine similarity between two vectors. Returns 0.0 if either vector is zero. This
def cosine_similarity(a, b):
    a = np.array(a); b = np.array(b)
    na = np.linalg.norm(a); nb = np.linalg.norm(b)
    if na == 0 or nb == 0: return 0.0
    return float(np.dot(a, b) / (na * nb))

# Softmax function to convert logits to probabilities. This is used in the sampling
def softmax(logits):
    m = max(logits)
    exps = [exp(l - m) for l in logits]
    s = sum(exps)
    return [e / s for e in exps]

# Top-p (nucleus) sampling function. It takes a list of candidate tokens, their probabilities,
# and a threshold p. It returns the smallest set of likely next tokens whose combined probability reaches p.
def top_p_sampling(cands, probs, p):
    paired = list(zip(cands, probs))
    paired.sort(key=lambda x: x[1], reverse=True)
    cum = 0.0; bucket = []
    for tok, pr in paired:
        bucket.append((tok, pr))
        cum += pr
        if cum >= p:
            break

```

```
toks, ps = zip(*bucket)
return random.choices(toks, weights=ps, k=1)[0]
```

Step 1 — Tokenization

Conceptually:

- Tokenization breaks text into discrete pieces (tokens) the model can process. Real tokenizers produce subwords or byte-pair encodings and map them to integer IDs.

In the code:

- We use the provided token list and token-to-ID mapping to mimic the tokenizer and vocabulary lookup. The next cell prints the tokens and their numeric IDs.

```
In [5]: # Step 1 - show tokens and IDs
tokens = list(tokenized_sentence_2)
token_ids = [token_to_id[t] for t in tokens]

print("Tokens:", tokens)
print("Token IDs:", token_ids)
```

```
Tokens: ['Lord', 'Jesus', 'Christ', 'Son', 'of', 'the', 'Living', 'God', ',', ',']
Token IDs: [102, 203, 458, 71, 5, 3, 890, 412, 12]
```

Step 2 — Embeddings

Conceptually:

- Each token ID is mapped to a dense vector (embedding) that captures semantic relationships. These vectors are the continuous inputs the model transforms with neural layers.

In the code:

- We print the toy 4-dimensional embeddings for each token and compute a simple sentence embedding by averaging token embeddings to demonstrate a compact context summary.

```
In [6]: # Step 2 - show token embeddings and sentence embedding
print("Token -> embedding:")

for t in tokens:
    print(f"{t:8s} -> {embedding_matrix[token_to_id[t]].tolist()}")

seq_embs = [embedding_matrix[token_to_id[t]] for t in tokens]
sentence_emb = np.mean(np.vstack(seq_embs), axis=0)
print("\nSentence embedding (mean):", sentence_emb.tolist())
```

Token -> embedding:

Lord -> [0.21, -0.11, 0.54, 0.03]
 Jesus -> [0.19, -0.09, 0.58, 0.01]
 Christ -> [0.22, -0.12, 0.6, 0.0]
 Son -> [0.05, 0.02, 0.1, 0.01]
 of -> [0.01, 0.0, 0.02, 0.0]
 the -> [0.02, -0.01, 0.01, 0.0]
 Living -> [0.18, -0.05, 0.3, 0.02]
 God -> [0.2, -0.1, 0.55, 0.02]
 , -> [0.0, 0.0, 0.0, 0.0]

Sentence embedding (mean): [0.12000000000000001, -0.05111111111111111, 0.30000000000000004, 0.010000000000000002]

Step 3 — Attention / Context

Conceptually:

- Attention computes which tokens in the context are most relevant to predicting the next token. True Transformer attention uses learned query/key/value projections and multi-head attention.

In the code:

- We use a very small toy attention: compute cosine similarity between each token embedding and the final token's embedding, normalize those similarities into weights, and form a context vector as a weighted sum. This demonstrates how attention converts multiple embeddings into a focused context vector for next-token prediction.

```
In [7]: # Step 3 - toy attention and context vector
last_emb = embedding_matrix[token_to_id[tokens[-1]]]

attn_scores = [cosine_similarity(last_emb, embedding_matrix[token_to_id[t]]) for t
attn_weights = np.array(attn_scores)

if attn_weights.sum() != 0:
    attn_weights = attn_weights / attn_weights.sum()
else:
    attn_weights = np.ones_like(attn_weights) / len(attn_weights)

print("Attention weights (token -> weight):")

for t, w in zip(tokens, attn_weights):
    print(f" {t:8s}: {w:.3f}")
context_vector = sum(w * embedding_matrix[token_to_id[t]] for t, w in zip(tokens, a
print("\nContext vector:", context_vector.tolist())
```

Attention weights (token -> weight):

```
Lord    : 0.111
Jesus   : 0.111
Christ  : 0.111
Son     : 0.111
of      : 0.111
the     : 0.111
Living  : 0.111
God     : 0.111
,       : 0.111
```

Context vector: [0.12, -0.05111111111111111, 0.3, 0.01]

Step 4 — From Context to Logits

Conceptually:

- The model converts its contextual representation into logits: unnormalized scores for each vocabulary token. In real models this is a learned linear projection from the model's last layer into vocabulary space.

In the code:

- We compute cosine similarity between the context vector and each candidate token embedding, scale those similarities to produce logits, and allow an optional `logit_bias` dictionary that adds a positive value to selected tokens' logits to nudge their probability up.

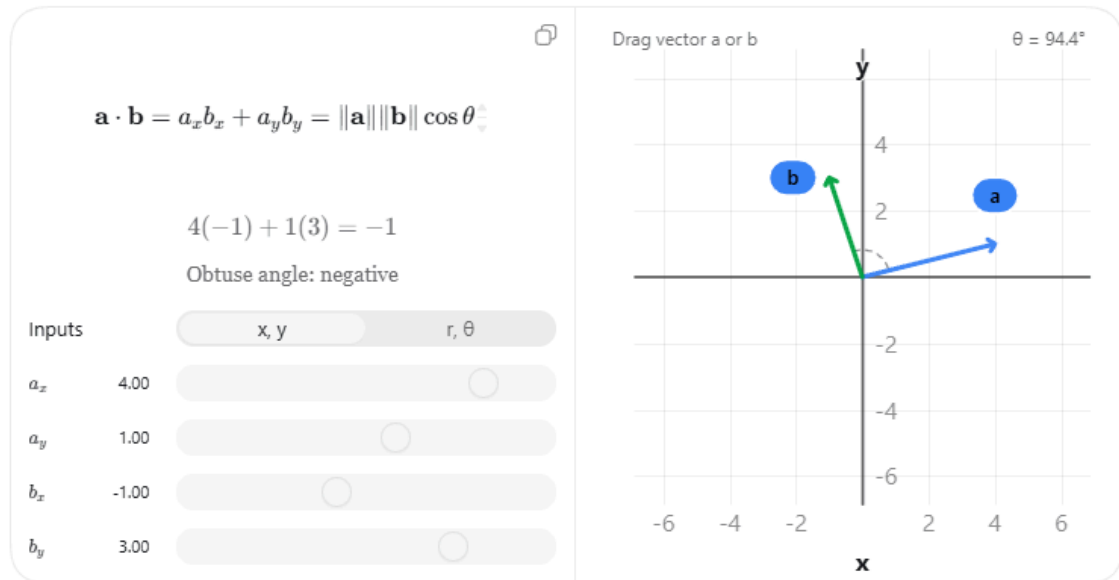
```
In [8]: # Step 4 - define compute_logits function and demonstrate with no bias
def compute_logits_from_context(ctx_vec, cand_ids, logit_bias=None, scale=10.0):
    sims = [cosine_similarity(ctx_vec, embedding_matrix[cid]) for cid in cand_ids]
    logits = [s * scale for s in sims]
    if logit_bias:
        for i, cid in enumerate(cand_ids):
            logits[i] += logit_bias.get(cid, 0.0)
    return logits

# Show logits with no bias
logits = compute_logits_from_context(context_vector, candidate_ids)

print("Candidate -> logit:")

for cid, l in zip(candidate_ids, logits):
    print(f" {id_to_token[cid]:7s} -> {l:.3f}")
```

Candidate -> logit:
 <STOP> -> 0.000
 reigns -> 9.996
 is -> 9.870
 Lord. -> 9.993
 blessed -> 9.966
 have -> 9.962
 mercy -> 9.968
 on -> 9.661
 us. -> 9.870



Step 5 — Logits → Probabilities and Decoding

Conceptually:

- Softmax converts logits to probabilities. Temperature scales logits to control randomness. Decoding strategies (greedy, top-k, top-p/nucleus, or sampling) determine how the next token is chosen.

In the code:

- We apply temperature scaling and a numerically stable softmax to convert logits into probabilities.
- We implement top-p (nucleus) sampling to pick the next token from the smallest set whose cumulative probability $\geq p$.

```
In [9]: # Step 5 - sampling from logits (demo)
def sample_next_token(context_vec, cand_ids, temperature=1.0, method="top-p", top_p
    logits = compute_logits_from_context(context_vec, cand_ids, logit_bias=logit_bi
    scaled_logits = [1 / max(1e-6, temperature) for l in logits]
    probs = softmax(scaled_logits)

    if method == "greedy":
```

```

        idx = int(np.argmax(probs)); return cand_ids[idx], probs
    elif method == "top-p":
        chosen = top_p_sampling(cand_ids, probs, p=top_p); return chosen, probs
    else:
        return random.choices(cand_ids, weights=probs, k=1)[0], probs

# Show probabilities and a sample pick
probs = softmax([1 / 0.8 for l in logits]) # example scaling

print("Candidate -> probability (example temp=0.8):")

for cid, p in zip(candidate_ids, probs):
    print(f" {id_to_token[cid]:7s} -> {p:.3f}")

chosen, _ = sample_next_token(context_vector, candidate_ids, temperature=0.8, method="top-p")
print("\nSampled next token id:", chosen, "token:", id_to_token[chosen])

```

Candidate -> probability (example temp=0.8):

```

<STOP> -> 0.000
reigns -> 0.138
is      -> 0.118
Lord.   -> 0.137
blessed -> 0.133
have    -> 0.132
mercy   -> 0.133
on       -> 0.091
us.     -> 0.118

```

Sampled next token id: 1000 token: reigns

Step 6 — Autoregressive Loop and Stop Condition

Conceptually:

- Autoregressive generation repeats: compute next-token distribution given the current prefix, choose a token, append it, and repeat until a stop token or max length is reached. The model generates incrementally — it never produces the entire answer all at once.

In the code:

- We run a loop that recomputes a simple context vector from the current tokens, samples the next token using top-p sampling with a logit bias that nudges the phrase "have mercy on us.", appends the chosen token to the sequence, and ends when the token is generated or the target phrase has been produced.

```

In [12]: # Step 6 - generation loop with logit-bias nudging "have mercy on us."
tokens = list(tokenized_sentence_2) # reset to original prefix
generated = []
max_steps = 12
temperature = 0.8
method = "top-p"

```

```

top_p = 0.9

# The target phrase, and a pointer to the word we want next
target_phrase = ["have", "mercy", "on", "us."]
target_pos = 0

for step in range(max_steps):
    # Defensive mapping for current ids
    current_ids = [token_to_id[t] for t in tokens if t in token_to_id]

    if not current_ids:
        print("No known token IDs in prefix. Stopping.")
        break

    seq_embs = [embedding_matrix[i] for i in current_ids]
    ctx_vec = np.mean(np.vstack(seq_embs), axis=0)

    # Bias only the NEXT word of the target phrase, so the words come out in order
    if target_pos < len(target_phrase):
        logit_bias = {token_to_id[target_phrase[target_pos]]: 8.0} # strong position bias
    else:
        logit_bias = {}

    next_id, probs = sample_next_token(ctx_vec, candidate_ids, temperature=temperature)
    next_tok = id_to_token[next_id]
    print(f"Step {step+1} chosen: '{next_tok}'")

    if next_tok == "<STOP>":
        print("Encountered <STOP>. Ending.")
        break

    # Append and ensure token_to_id mapping exists for new token
    tokens.append(next_tok)
    token_to_id[next_tok] = next_id
    generated.append(next_tok)

    # Advance to the next target word; stop once the full phrase appears
    if target_pos < len(target_phrase) and next_tok == target_phrase[target_pos]:
        target_pos += 1
        if target_pos == len(target_phrase):
            print("Target phrase produced. Stopping.")
            break

print("\nGenerated continuation:", " ".join(generated))
print("Full final sequence:", " ".join(tokens))

```

Step 1 chosen: 'have'
 Step 2 chosen: 'mercy'
 Step 3 chosen: 'on'
 Step 4 chosen: 'us.'
 Target phrase produced. Stopping.

Generated continuation: have mercy on us.
 Full final sequence: Lord Jesus Christ Son of the Living God , have mercy on us.

3.2 How do Google Searches Work With AI?

Here's a clear, step-by-step explanation of how Google produces those real-time-looking answers.

High-level overview - Google does not perform a full live crawl of the web every time you search. Instead, it continuously crawls and indexes huge parts of the web ahead of time, then serves results from that index very quickly. For AI-style direct answers, Google combines that indexed content with retrieval systems and on-the-fly synthesis.

Detailed steps

1. Crawling: finding pages

- Googlebot (and other crawlers) visit billions of URLs on a schedule. Crawlers follow links and sitemaps, discover new pages, and re-check existing pages. Frequency depends on the site's importance and how often content changes.

2. Indexing: storing what's discovered

- Crawled page content is parsed and stored in Google's index. The index is a compressed, searchable representation of page content (text, metadata, structured data). The index also stores signals for ranking (e.g., page structure, mobile-friendliness, speed).

3. Ranking & retrieval: deciding what's relevant

- When you type a query, Google quickly matches it against the index to retrieve candidate documents and passages. Ranking algorithms (hundreds of signals) score candidates for relevance, quality, and usefulness. Machine-learned models (e.g., BERT, MUM, later models) help understand query intent and document semantics.

4. Generating the answer you see

- For classic results: Google ranks and displays links, snippets, and rich features (knowledge panels, carousels).
- For instant answers / featured snippets: Google extracts or synthesizes a short passage from a high-scoring page and displays it at the top. That passage usually comes directly from an indexed page or structured data on that page.
- For AI-styled answers (e.g., "Google's summarized answer"): Google often uses retrieval-augmented methods: it retrieves relevant passages from the index, then uses a generative model to synthesize a concise reply grounded in those passages. The model is not performing a live crawl — it's reading text that was already indexed, often very recently crawled. Google may also attach citations/links to the source passages shown.

5. Freshness & "real-time" behavior

- Some pages are crawled and re-indexed frequently (news, high-traffic pages), so the index is often very up-to-date. For time-sensitive searches, Google has mechanisms to prioritize freshly crawled content and real-time signals (news feeds, publisher push notifications, APIs).
- For behind-the-scenes real-time content (stock tickers, live sports), Google pulls data from specific APIs or licensed data feeds rather than doing a fresh full crawl.

6. Signals that improve answer quality

- Structured data (schema.org), sitemaps, canonical tags and high-quality on-page content make a page easier to index and cite.
- User-engagement signals, spam detection, and manual review can also influence which content is used for featured snippets or AI answers.

Why it looks “live”: Because Google’s crawling and indexing are continuous, many pages in the index are recent. Combined with sophisticated retrieval and synthesis, the results can appear as if Google did a live web crawl for your query — but in reality the heavy lifting usually happened earlier and at scale.

- Limitations and caveats
 - The model or snippet might be based on outdated content if a page changed after the last crawl.
 - Summaries can omit nuance or context; always check the cited source for important decisions.
 - For very new, low-coverage, or private content (paywalled or behind authentication), Google’s index may not contain the needed pages.
- Short example workflow (query → answer)
 1. You search “latest recall on [product].”
 2. Google retrieves recent pages about that product from its index.
 3. It ranks pages and extracts key passages.
 4. If it can, it shows a featured snippet or an AI-synthesized answer based on those passages, and provides links to the source pages (often with timestamps).
- Bottom line
 - Google’s answers combine a large pre-built, continuously updated index (from prior crawls) with fast retrieval, ranking, and occasionally on-the-fly synthesis. It rarely does a full live crawl per query; instead it serves answers from its massive, frequently refreshed index and targeted real-time feeds when necessary.

3.3 Where do LLMs get their data?

Training data for LLMs consists of vast collections of text, often terabytes in size, gathered from public sources like websites, online books, research papers, and code repositories.

Specifically, the major sources include:

- **Web Crawls** — The largest portion of LLM training data comes from publicly accessible text aggregated into massive datasets, such as Common Crawl, a nonprofit initiative that crawls the web and archives billions of web pages annually. This includes blogs, forums, news sites, and more.
- **Wikipedia** — A free, crowdsourced encyclopedia with 60M+ articles in 300+ languages. Its structured, verified content helps LLMs learn factual information. Academic repositories like arXiv and PubMed provide scientific texts, while platforms like Reddit and Stack Overflow contribute informal, conversational data.
- **Books & Literature** — Commonly used sources also include video transcripts, online publications, research articles, and code archives — together providing a broad representation of human language and knowledge.
- **Code Repositories** — For coding LLMs like Codex or StarCoder, public code from platforms like GitHub is used to train models to generate and understand programming languages.

During training, an LLM is fed a huge amount of text data from various sources, which effectively teaches the model about language and facts in the world. The training process adjusts the model's internal parameters so that it can predict the next word in a sentence, form coherent paragraphs, and capture relationships between concepts.

Importantly, LLMs don't store data like a database of facts with easy retrieval. Instead, through training, they compress information into the weights (parameters) of the model.

Think of it like absorbing millions of books and being able to recall the *gist* — not a word-for-word copy.

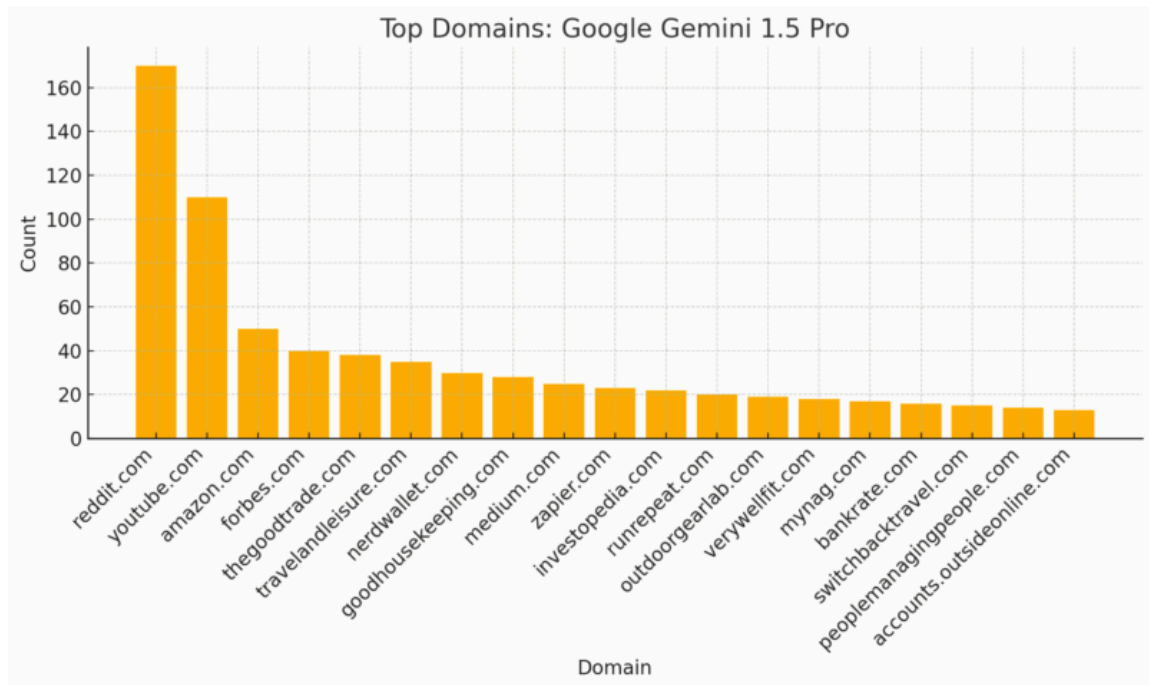
Platforms like ChatGPT, Perplexity AI, Google Gemini, and Claude are rapidly becoming primary interfaces for discovery and research. Unlike traditional search engines that return a list of links, these AI systems synthesize answers — they don't just show results, they decide which sources are credible enough to include in a response.

Only about 14% of top domains overlap across AI platforms, meaning ChatGPT, Gemini, and Perplexity do not rely on the same sources.

3.3.1 Where Google Gemini 1.5 Pro Got Its Training Data

	Training Data	Cited Domains (Your Chart)
What it is	Data the model <i>learned from</i> during training	Domains the model <i>references</i> in its answers

	Training Data	Cited Domains (Your Chart)
When it happens	Before deployment	At runtime, when you ask it a question
Example	Common Crawl, Wikipedia, Books	reddit.com, youtube.com, amazon.com
Purpose	Builds the model's knowledge	Grounds and supports the model's responses



<https://martech.org/where-do-the-popular-llms-find-their-information/>

4. Common Misconceptions

Here are some common misconceptions students often have after an introduction to deep learning and Large Language Models:

1. **"AI understands like a human."** Students often assume an LLM truly *understands* meaning, emotions, or truth the way people do. In reality, it predicts patterns in language based on training data.
2. **"ChatGPT searches the internet for every answer."** Many students think the model is constantly Googling information. Most LLM responses are generated from patterns learned during training, not live web searches.
3. **"The AI remembers everything forever."** Students may think the model permanently remembers every conversation or fact. Most interactions are temporary unless memory features are specifically enabled.

4. **“Neural networks think exactly like the human brain.”** Artificial neural networks were inspired by biology, but they are mathematical systems, not digital brains.
5. **“Bigger models are always smarter.”** Larger models often perform better, but size alone does not guarantee accuracy, reasoning quality, or truthfulness.
6. **“AI knows when it is wrong.”** LLMs can produce very confident incorrect answers (“hallucinations”) without realizing they are inaccurate.
7. **“Training means programmers type in all the answers.”** Students sometimes imagine humans manually entering responses. Instead, models learn statistical patterns from enormous datasets.
8. **“AI is objective and unbiased.”** Because models are trained on human-created data, they can inherit biases present in society or the training material.
9. **“Deep learning is magic.”** Students may see AI as mysterious rather than mathematical. Emphasizing layers, weights, probabilities, and optimization helps demystify it.
10. **“LLMs are databases.”** Students often think the model stores and retrieves exact answers like a filing cabinet. Instead, it generates likely sequences of tokens based on learned relationships.
11. **“If the answer sounds good, it must be true.”** Fluent language can create a false sense of authority. Students need practice verifying claims independently.
12. **“AI will completely replace human thinking.”** Students may swing toward either hype or fear. It helps to frame AI as a powerful tool with strengths and weaknesses rather than an all-knowing replacement for humans.

5. Exit Ticket

1. What is the main difference between traditional programming and machine learning?
2. Why is deep learning called “deep”?
3. What is an artificial neural network loosely inspired by?
 - Human memory
 - The human brain
 - The internet
 - Computer chips
4. Explain what happens during training when a neural network makes mistakes.

5. True or False: Large Language Models actually “understand” language the same way humans do.
6. What is one task that deep learning systems are especially good at?
 - Image recognition
 - Speech recognition
 - Language translation
 - All of the above
7. What is a “hallucination” in an AI system?
8. Why can AI systems sometimes contain bias?
9. True or False: ChatGPT searches the live internet for every answer it gives.
10. In one or two sentences, describe one benefit and one risk of deep learning or LLMs.

6. Vocabulary

1. **Large Language Model (LLM)** — A type of AI trained on massive amounts of text to generate human-like responses by predicting the next word or token.
2. **Token** — A small unit of text processed by an AI model, such as a word, part of a word, or punctuation mark.
3. **Training Data** — The books, articles, websites, and other digital text used to teach an AI model language patterns.
4. **Web Scraping** — The automated collection of information from websites that can become part of an AI model’s training data.
5. **Deep Learning** — A branch of machine learning that uses many layers of neural networks to learn complex patterns from data.
6. **Neural Network** — A system of interconnected mathematical “neurons” that processes information and learns patterns.
7. **Backpropagation** — A process where a neural network learns by measuring mistakes and adjusting itself to improve accuracy.
8. **Pattern Recognition** — The ability of AI systems to identify relationships, trends, and structures in data.
9. **Hallucination** — When an AI system produces incorrect or fictional information that sounds believable.

10. **Inference** — The process of using a trained AI model to generate answers, predictions, or responses.

© 2026 D. E. Goddu. All rights reserved. Developed by the author with the assistance of AI tools (Claude, Anthropic). All content was selected, reviewed, and edited by the author.